

Setup Guide

1. Clone the Repository

```
cd /common/users/$USER
git clone -b adaptive_main https://github.com/dhruvmetha/adaptive_roa.git
cd adaptive_cartpole
```

2. Create Conda Environment

On Westeros/GPU machines:

```
# Use system conda
source /koko/system/anaconda/etc/profile.d/conda.sh

# Create environment from yaml (if available) or manually
conda create -p /common/users/$USER/miniforge3/envs/adaptive_roa
python=3.10 -y
conda activate /common/users/$USER/miniforge3/envs/adaptive_roa

# Install PyTorch with CUDA support
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu118

# Install other dependencies
pip install pytorch-lightning hydra-core omegaconf
pip install numpy scipy matplotlib tqdm
pip install flow-matching # Facebook's flow matching library

# Install the project in development mode
pip install -e .
```

On iLab machines:

```
# iLab uses a different conda path
source /common/users/$USER/miniconda3/etc/profile.d/conda.sh
# Or if using miniforge:
source /common/users/$USER/miniforge3/etc/profile.d/conda.sh

conda activate /common/users/$USER/miniforge3/envs/adaptive_roa
```

3. Set Up Environment Variables

Create a `.env` file in the project root:

```
cd /common/users/$USER/adaptive_cartpole
cat > .env << EOF
NET_ID=$USER
EOF
```

This file is used by Hydra configs to resolve paths like `${net_id}`.

4. Verify Data Access

The trajectory data is stored in a shared location:

```
# Check that you can access the shared data
ls /common/users/shared/pracsys/genMoPlan/data_trajectories/

# You should see directories like:
# - cartpole_pybullet/
# - mountain_car_power_0p0008/
# - pendulum_lqr_50k/
```

Each system directory contains:

- `trajectories/` - Individual trajectory files (sequence_*.txt)
 - `train_test_splits/` - Shuffled indices and labels for training
 - `roa_labels.txt` - Full dataset labels for ROA evaluation
-

5. Verify GPU Access

```
# Check available GPUs
nvidia-smi

# Request a GPU interactively (if using SLURM)
srun -G 1 --pty /bin/bash
```

6. Run a Test

Quick import test:

```
cd /common/users/$USER/adaptive_cartpole
export PYTHONPATH=$(pwd):$PYTHONPATH

python -c "
from src.adaptive.data_source import TrajectoryDataSource,
TrajectoryDataSourceConfig
from src.systems.cartpole import CartPoleSystem
print('All imports successful!')
"
```

Run adaptive sampling (CartPole):

```
# Set up environment
source /koko/system/anaconda/etc/profile.d/conda.sh
conda activate /common/users/$USER/miniforge3/envs/adaptive_roa
cd /common/users/$USER/adaptive_cartpole
export PYTHONPATH=$(pwd):$PYTHONPATH

# Run with default config
CUDA_VISIBLE_DEVICES=0 python src/adaptive/run_adaptive_cartpole.py

# Run with custom parameters
CUDA_VISIBLE_DEVICES=0 python src/adaptive/run_adaptive_cartpole.py \
    initial_train_size=1000 \
    n_epochs=1 \
    trainer.max_epochs=500 \
    batch_size=1024
```

Run adaptive sampling (Mountain Car):

```
CUDA_VISIBLE_DEVICES=0 python src/adaptive/run_adaptive_mountain_car.py \
    initial_train_size=1000 \
    n_epochs=1 \
    trainer.max_epochs=500 \
    batch_size=512
```

Run adaptive sampling (Pendulum):

```
CUDA_VISIBLE_DEVICES=0 python src/adaptive/run_adaptive_pendulum.py \
    initial_train_size=500 \
    n_epochs=1 \
    trainer.max_epochs=500
```

7. Running Multiple Experiments

Using different shuffle variants (for cross-validation):

```
# Run with shuffle variant 0 (default)
python src/adaptive/run_adaptive_cartpole.py shuffle_variant=0

# Run with shuffle variant 3
python src/adaptive/run_adaptive_cartpole.py shuffle_variant=3

# Run all 10 variants
for i in {0..9}; do
    CUDA_VISIBLE_DEVICES=$((i % 4)) python
src/adaptive/run_adaptive_cartpole.py \
    shuffle_variant=$i &
done
wait
```

Running in background with logging:

```
mkdir -p logs

CUDA_VISIBLE_DEVICES=0 python src/adaptive/run_adaptive_cartpole.py \
    > logs/cartpole_run.log 2>&1 &

# Monitor progress
tail -f logs/cartpole_run.log
```

8. Key Configuration Parameters

Edit `configs/adaptive_cartpole_pybullet.yaml` (or pass as CLI args):

Parameter	Description	Default
<code>initial_train_size</code>	Initial training trajectories	1000
<code>n_epochs</code>	Number of adaptive sampling epochs	1
<code>warm_start</code>	Continue training from previous checkpoint	true
<code>adaptive_data_max</code>	New samples per epoch	50
<code>d2_ratio</code>	Fraction for uncertainty-filtered sampling	0.5
<code>trainer.max_epochs</code>	Training epochs per round	2000
<code>batch_size</code>	Training batch size	512
<code>optimizer.lr</code>	Learning rate	1e-3

Parameter	Description	Default
<code>conformal.optimize_mode</code>	"lambda" or "delta"	delta
<code>shuffle_variant</code>	Data shuffle variant (0-9)	0

9. Output Structure

Runs output to `outputs/<system>/<timestamp>/`:

```

outputs/adaptive_cartpole_pybullet/2026-01-18_16-45-00/
├── .hydra/                # Hydra config snapshots
├── datasets/              # Built endpoint datasets
│   ├── train_endpoint_dataset.txt
│   ├── val_endpoint_dataset.txt
│   └── test_endpoint_dataset.txt
├── epoch_0/               # Per-epoch outputs
│   ├── checkpoints/      # Model checkpoints
│   └── version_0/        # Lightning logs
├── conformal_results.json # Conformal prediction results
└── roa_evaluation.json    # Full ROA evaluation metrics

```

10. Troubleshooting

`ModuleNotFoundError: No module named 'src'`

```
export PYTHONPATH=/common/users/$USER/adaptive_cartpole:$PYTHONPATH
```

CUDA out of memory

Reduce batch size:

```
python src/adaptive/run_adaptive_cartpole.py batch_size=256
```

Hydra override errors

Use `+` prefix for new keys, no prefix for existing keys:

```

# Override existing key
python script.py batch_size=512

# Add new key (not in config)
python script.py +new_param=value

```

conda: command not found

```
# On westeros
source /koko/system/anaconda/etc/profile.d/conda.sh

# On ilab
source /common/users/$USER/miniconda3/etc/profile.d/conda.sh
```

11. Quick Reference Commands

```
# Activate environment (westeros)
source /koko/system/anaconda/etc/profile.d/conda.sh
conda activate /common/users/$USER/miniforge3/envs/adaptive_roa
cd /common/users/$USER/adaptive_cartpole
export PYTHONPATH=$(pwd):$PYTHONPATH

# Check GPU status
nvidia-smi

# Run CartPole
CUDA_VISIBLE_DEVICES=4 python src/adaptive/run_adaptive_cartpole.py

# Run Mountain Car
CUDA_VISIBLE_DEVICES=5 python src/adaptive/run_adaptive_mountain_car.py

# Run Pendulum
CUDA_VISIBLE_DEVICES=6 python src/adaptive/run_adaptive_pendulum.py

# Monitor logs
tail -f logs/*.log

# Check running jobs
ps aux | grep python
```